```
client 2 writes, Nagle on          p50   41.00  p99   41.34 ms
client 2 writes, TCP_NODELAY       p50    0.04  p99    0.09 ms
```

A companion to the Testing Field Guide

Java *performance* deep dives

TCP_NODELAY and delayed ACKs, connections and timeouts, pools and Little's Law, measuring latency, the JIT, GC pauses, virtual threads, containers and the memory model — each explained from the specification or the source code, and each measured. Every number in this book is read from a log written in CI.

10	19	25	1
chapters	experiments	logs	runner, named

INSIDE

Contents

How it was measured — the runner, the experiments, the rules 03

01 **Nagle, delayed ACK and TCP_NODELAY**
The mechanism, packet by packet 04
Measured; who sets TCP_NODELAY 05

02 **Connections**
A connection's life 06
Measured: reuse and close 07

03 **Timeouts**
Phases and defaults 08
Measured: three broken peers 09

04 **Pools and Little's Law**
Little's Law 10
Measured: database and CPU pools 11

05 **Measuring latency**
Open and closed loops 12
Measured: the same server, three ways 13

06 **JIT and benchmarks**
Warm-up 14
Dead code and constant folding, with JMH 15

07 **GC pauses**
Pauses and the G1 cycle 16
Measured: the tail 17

08 **Virtual threads**
Measured: blocking and pinning 18

09 **Containers and CPUs**
What the JVM sees 19
Sized from the CPU count; throttling 20

10 **The Java Memory Model**
What another thread may see 21
Visibility and lost updates 22

Sources — every specification, source file and document cited 23
Index A–Z 24

Facts as of October 2026, for JDK 21 unless a page says otherwise. Every entry, page reference and footer link is clickable.

COLOUR LANGUAGE

Network · queues and measurement · the JVM · the platform

READ WITH

Testing Java applications — Field Guide (Testing-Field-Guide.pdf, same repository). Its performance chapter introduces several of these topics in a page each; this book is the long version.

ON EVERY PAGE

The mechanism, from the specification or the source. The experiment's code, as it ran. The real output. What to do about it. The sources, at the foot.

BEFORE YOU START

How it was measured

This book makes two kinds of statement: how something works, and what it did when we ran it. Both come with evidence. Nothing is taken from blog posts or from memory.

How it works: primary sources only

The specification (RFC 9293 for TCP, the Java Language Specification for the memory model, JEPs for JDK features), the **source code of the version discussed** — OpenJDK 21, the Linux kernel, the libraries — or the vendor's own documentation. Each page lists what it used at its foot; the full list is at the back.

What it did: an experiment, run in CI

Each chapter has a small program in `deepdive/`. The `deepdive` CI job runs all of them on every push (`deepdive/run.sh`) and publishes the logs to the `ci-logs-deepdive` branch. This book is built from those logs: every measured number on a page is read from them at build time.

The machine

Runner	GitHub Actions, `ubuntu-latest`
CPU	4 CPUs · INTEL(R) XEON(R) PLATINUM 8573C
Kernel	Linux 6.17.0-1022-azure, `CONFIG_HZ=1000`
JDK	OpenJDK 21.0.12.1 (Temurin)
`ip_local_port_range`	32768 60999
`tcp_tw_reuse`	2
`tcp_syn_retries`	6
`tcp_pingpong_thresh`	1
`tcp_fin_timeout`	60

From `machine.txt`, written by the same run.

The experiments

CH.	PROGRAM	LOG
1	tcp/Nagle.java + tcpdump	nagle.txt, nagle-trace.txt
2	tcp/Connections.java	connections.txt
3	tcp/Timeouts.java, SynRetries.java + iptables	timeouts.txt, syn*.txt
4	pool/PoolSize.java, CpuPool.java	pool.txt, cpupool.txt
5	latency/CoordinatedOmission.java	co.txt
6	jit/WarmUp.java, DeadCodeBench.java (JMH)	warmup*.txt, jmh.txt, jit-flags.txt
7	gc/GcLatency.java, G1 and ZGC	gc-*.txt, gc-*.log
8	vt/VirtualThreads.java	vt.txt
9	box/Resources.java, Throttle.java in Docker	containers.txt, throttle.txt
10	jmm/StaleFlag.java + 4 jcstress tests	staleflag.txt, jcstress.log

One number, traced

STEP	EVIDENCE
The claim (chapter 1)	Two small writes with Nagle on: median **41.00 ms** per request
The log line	`client 2 writes, Nagle on p50 41.00` — `nagle.txt`, written by `Nagle.java` in CI
The packets	`nagle-trace.txt`: the server's ACK of the first 50-byte half left 40.09 ms after it; the second half was held until then — tcpdump on loopback
Why 40 ms	Linux `include/net/tcp.h`: `TCP_DELACK_MIN = HZ/25`; with `CONFIG_HZ=1000` that is 40 ms
Why it waits at all	RFC 9293 §3.7.4 (Nagle: hold small segments while data is unacknowledged) and §3.8.6.3 (delayed ACK)

- **The text follows the data.** Where a sentence interprets a result — "slower", "removed by the JIT", "not a GC pause" — it is computed from the log, and changes if the result changes.
- **Conclusions are checked.** `guide/claims_deep.py` re-reads the logs and fails the build if a conclusion this book states no longer holds.
- **Real output is printed as it ran** — packet traces, GC logs, JMH and jcstress tables — not retyped.
- **One machine.** Absolute numbers depend on the CPU, kernel and JDK; the mechanisms and the ratios are what carry over. Rerun on yours: `bash deepdive/run.sh` (Linux, JDK 21, Maven, Docker; `sudo` for tcpdump and iptables).
- **Versions matter.** Statements are for JDK 21 and the library versions named; where a later JDK changed the behaviour (JEP 474, 490, 491) the page says so.

01 · NAGLE AND DELAYED ACK

Two good ideas that wait for each other

TCP has two old optimisations against tiny packets. The **sender** holds back a small segment while earlier data is unacknowledged (Nagle). The **receiver** holds back its ACK, hoping to piggyback it on a reply (delayed ACK). In a request/response exchange where one message goes out in two `write()` calls, each waits for the other — until a timer fires.

Nagle's algorithm — the sender

> "If there is unacknowledged data (i.e., SND.NXT > SND.UNA), then the sending TCP endpoint buffers all user data (regardless of the PSH bit) until the outstanding data has been acknowledged or until the TCP endpoint can send a full-sized segment." — RFC 9293 §3.7.4

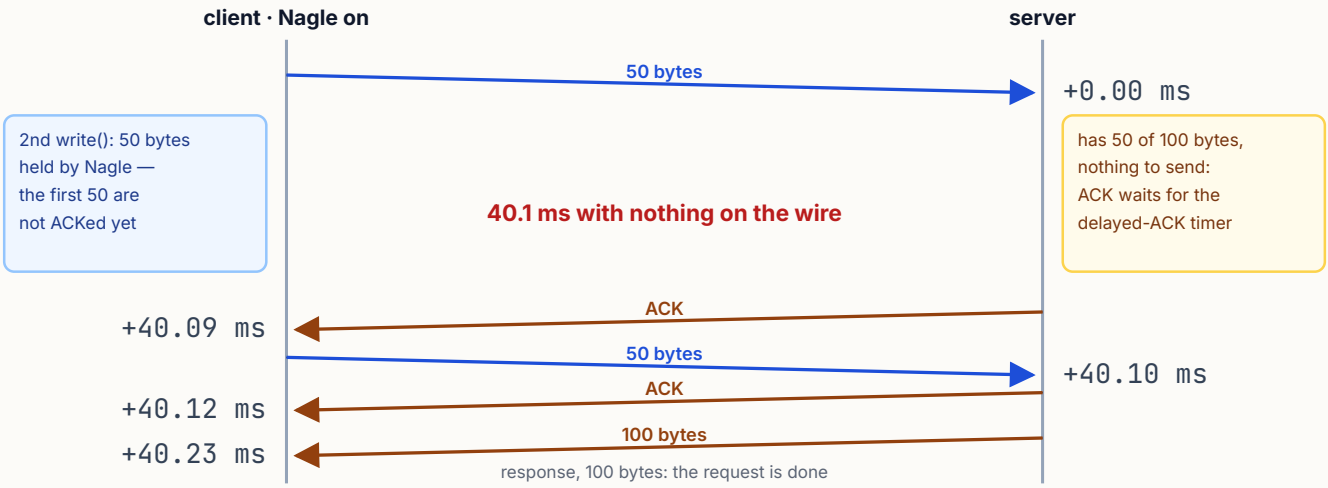
TCP **SHOULD** implement it (SHLD-7); there **MUST** be a way to disable it per connection (MUST-17). In Java that is `TCP_NODELAY`, off by default on a `java.net.Socket`.

Delayed ACK — the receiver

> "A TCP endpoint SHOULD implement a delayed ACK (SHLD-18) ... the delay MUST be less than 0.5 seconds (MUST-40)." An ACK SHOULD go out for at least every second full-sized segment. — RFC 9293 §3.8.6.3

Linux starts each connection in **quick-ACK** mode and leaves it once the connection looks like request/response ("ping-pong"): a reply sent within the ACK timeout of receiving data counts. From then on, a lone segment waits for the timer.

The stall, captured



LINUX	VALUE	MEANING	WHERE
`TCP_DELACK_MIN`	HZ/25 jiffies = **40 ms**	shortest delayed-ACK timer	include/net/tcp.h
`TCP_DELACK_MAX`	HZ/5 jiffies = **200 ms**	longest delayed-ACK timer	include/net/tcp.h
`TCP_MAX_QUICKACKS`	**16**	ACKs sent at once in quick-ACK mode	include/net/tcp.h
`tcp_pingpong_thresh`	**1** on the runner	replies before a connection counts as request/response	ip-sysctl.rst
`CONFIG_HZ`	**1000** on the runner	timer tick: 1 ms	/boot/config

The capture shows the second request on the connection. **40.1 ms** = the 40 ms minimum delayed-ACK timer plus timer granularity (1 ms ticks on this kernel).

The **first** request did not wait: its ACK came back in 0.017 ms, while the new connection was still in quick-ACK mode. After one request and its reply, Linux treats the connection as ping-pong and starts delaying ACKs.

SOURCES RFC 9293 (2022) §3.7.4, §3.8.6.3 · RFC 896 (Nagle, 1984) · Linux include/net/tcp.h, net/ipv4/tcp_input.c, tcp_output.c (tcp_event_data_sent: pingpong) · Documentation/networking/ip-sysctl.rst · OpenJDK 21 StandardSocketOptions.TCP_NODELAY · capture: tcpdump on loopback, GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

01 · NAGLE AND DELAYED ACK

Measured: 200 requests, 7 ways to send them

One connection on loopback, 200 request/response round trips of 100 bytes each way. The only differences: whether a message is written in one `write()` or two (like headers, then body), on which side, and which socket option is set.

Nagle.java — the client side

```java
static long[] exchange(Socket socket, int writes, int rounds)
    throws Exception {
  OutputStream out = socket.getOutputStream();
  InputStream in = socket.getInputStream();
  long[] rtt = new long[rounds];
  for (int i = 0; i < rounds; i++) {
    long t0 = System.nanoTime();
    write(out, writes);
    in.readNBytes(100);
    rtt[i] = System.nanoTime() - t0;
  }
  return rtt;
}

/** 100 bytes in one write, or 50 + 50 like headers + body. */
static void write(OutputStream out, int writes)
    throws Exception {
  if (writes == 1) {
    out.write(new byte[100]);
  } else {
    out.write(new byte[50]);
    out.write(new byte[50]);
  }
  out.flush();
}
```

SETUP	P50 MS	P99 MS	MAX MS	OVER 30 MS
client 2 writes, Nagle on	41.00	41.34	41.89	199 of 200
client 2 writes, TCP_NODELAY	0.04	0.09	0.46	0 of 200
client 1 write, Nagle on	0.03	0.06	0.15	0 of 200
server 2 writes, Nagle on	41.00	41.23	41.33	199 of 200
server 2 writes, TCP_NODELAY	0.04	0.07	0.26	0 of 200
client 2 writes, QUICKACK once	41.00	41.27	41.27	199 of 200
client 2 writes, QUICKACK per read	0.04	0.12	0.24	0 of 200

Real output · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

Who turns Nagle off for you

LIBRARY	TCP_NODELAY	EVIDENCE IN ITS SOURCE
JDK `HttpClient`	on	`PlainHttpConnection`: `setOption(TCP_NODELAY, true)`
JDK `HttpServer`	**off**	only with `-Dsun.net.httpserver.nodelay=true` (`ServerConfig`)
`java.net.Socket`	**off**	`StandardSocketOptions.TCP_NODELAY`: default `FALSE`
Tomcat	on	`SocketProperties.tcpNoDelay = TRUE`
Jetty 12	on	`ServerConnector._acceptedTcpNoDelay = true`
Netty 4.1	on	`DefaultSocketChannelConfig`: on unless Android
PostgreSQL JDBC	on	`PGProperty.TCP_NO_DELAY`: default `"true"`
Lettuce (Redis)	on	`SocketOptions.DEFAULT_SO_NO_DELAY = true`
Kafka client	on	`Selector`: `socket.setTcpNoDelay(true)`

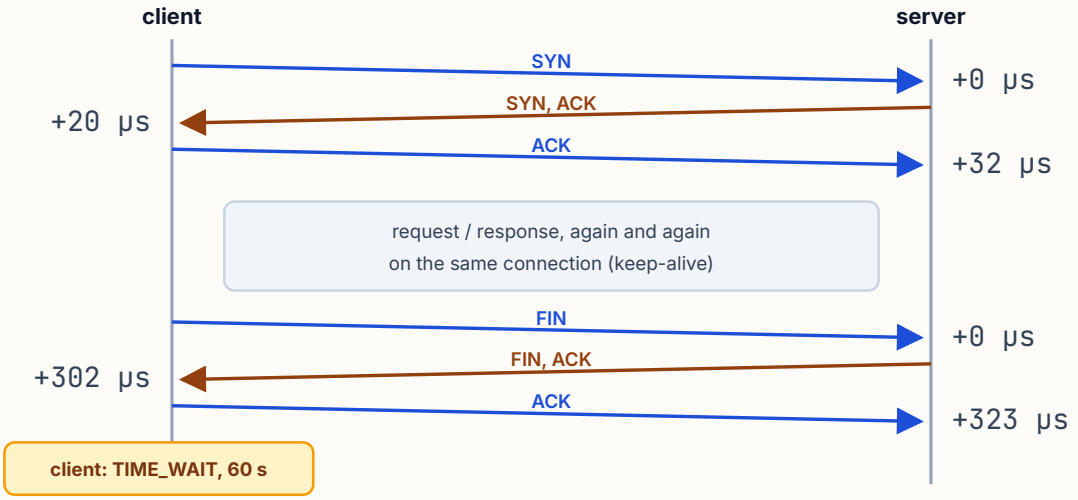
- **Two writes per message + Nagle = one timer per message.** Every round trip took about 41 ms; every fix brought p99 under 0.1 ms.
- **Write each message once.** Build it in a buffer (or wrap the stream in a `BufferedOutputStream`) and flush once: Nagle never has a second small segment to hold.
- **Or set `TCP_NODELAY`** on request/response connections — on the side that writes twice. Most servers and the JDK client already do; the JDK's own `HttpServer` does not.
- **The receiver can help:** `jdk.net.ExtendedSocketOptions.TCP_QUICKACK` (Linux, JDK 10+) asks for an immediate ACK — but it is not sticky. Set once, nothing changed; set before every read, the stall was gone.
- **Test for it:** a latency test on loopback shows it at once — on Linux, a p50 just over 40 ms with idle CPUs. The main guide's load test found exactly this in the JDK `HttpServer`.

SOURCES OpenJDK 21: jdk/internal/net/http/PlainHttpConnection.java, sun/net/httpserver/ServerConfig.java, ServerImpl.java, java/net/StandardSocketOptions.java, jdk/net/ExtendedSocketOptions.java (TCP_QUICKACK, since 10) · Tomcat main: SocketProperties.java · Jetty 12.0: ServerConnector.java · Netty 4.1: DefaultSocketChannelConfig.java · pgjdbc: PGProperty.java · Lettuce: SocketOptions.java · Kafka: clients/.../network/Selector.java · results: deepdive/logs/nagle.txt

02 · CONNECTIONS

Open once, reuse, close carefully

Before the first byte of a request, TCP spends a round trip on the handshake. After the last byte, the side that closes **first** keeps the connection in TIME_WAIT, holding its port. Both costs are paid per connection — so the number of connections, not requests, is what hurts.



TIME_WAIT

> "When a connection is closed actively, it MUST linger in the TIME-WAIT state for a time 2xMSL (Maximum Segment Lifetime) (MUST-13)." "For this specification the MSL is taken to be 2 minutes." — RFC 9293 §3.6.1, §3.4.2

Sequence numbers stay "busy" until MSL seconds have passed (§3.4.3): the wait keeps old segments from being taken for a new connection's. Linux does not wait 4 minutes but a fixed **60 s** (`TCP_TIMEWAIT_LEN`). `tcp_fin_timeout` is something else — how long an orphaned connection may sit in FIN_WAIT_2.

Ephemeral ports run out

A connection is identified by four values: source address and port, destination address and port. To one destination, a client can only vary its own port. The runner's range is `32768–60999` : **28,232** ports. Each closed connection keeps its port for 60 s, so one client address can open at most about **471 new connections per second** to one server address and port, for as long as that lasts — unless TIME_WAIT reuse applies.

Reuse is not on everywhere

`tcp_tw_reuse` lets a new outgoing connection take a port still in TIME_WAIT when that is safe. Its default is **2: loopback traffic only** — the runner has `2` . So a load test against `localhost` can open connections endlessly, while the same test against a real host runs out of ports. Test against the network you will run on.

States you will see in `ss` , and what they tell you

STATE	MEANS	MANY OF THEM USUALLY MEAN
`SYN-SENT`	connect() sent a SYN, no answer yet	the server is unreachable or dropping packets — see the next chapter
`ESTABLISHED`	open, data can flow both ways	normal; compare with your pool sizes
`FIN-WAIT-2`	we closed; the peer has not closed its side yet	the peer is not closing its sockets
`CLOSE-WAIT`	the peer closed; our application has not called close()	**a socket leak in our code**
`TIME-WAIT`	we closed first; waiting before the port can be reused	many short-lived connections: reuse them

SOURCES RFC 9293 §3.3.2, §3.4.2, §3.4.3, §3.6.1 · Linux include/net/tcp.h (TCP_TIMEWAIT_LEN) · Documentation/networking/ip-sysctl.rst (ip_local_port_range, tcp_tw_reuse, tcp_fin_timeout) · capture and sysctl values: GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

02 · CONNECTIONS

Measured: 2,000 requests, five ways

2,000 request/response exchanges on loopback: over one kept-alive connection, then over a new connection each time, closed four different ways. Afterwards the program counts sockets by state in the kernel's own tables, `/proc/net/tcp` and `tcp6`.

HOW	MS / REQUEST	VS KEPT-ALIVE	TIME_WAIT CLIENT / SERVER	CLOSE_WAIT CLIENT / SERVER	FIN_WAIT2 CLIENT / SERVER
one kept-alive connection	40.8	1.0×	–	–	–
new connection, client closes first	196.5	4.8×	**2000** / 0	0 / 0	0 / 0
new connection, server closes first	163.1	4.0×	0 / **2000**	0 / 0	0 / 0
new connection, client closes with `SO_LINGER(0)`	183.3	4.5×	0 / 0	0 / 0	0 / 0
new connection, server never closes (a leak)	164.3	4.0×	0 / 0	0 / **2000**	**2000** / 0

Real output · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

Connections.java — a new connection per request

```
static void perRequest(Close how) throws Exception {
  try (var server = new ServerSocket(0, 512, LO)) {
    int port = server.getLocalPort();
    Thread.ofPlatform().daemon()
        .start(() -> oneEach(server, how));
    long t0 = System.nanoTime();
    for (int i = 0; i < N; i++) {
      try (var s = new Socket(LO, port)) {
        s.setTcpNoDelay(true);
        if (how == Close.CLIENT_RESET) {
          s.setSoLinger(true, 0);       // close() sends RST
        }
        s.getOutputStream().write(new byte[100]);
        var in = s.getInputStream();
        in.readNBytes(100);
        if (how == Close.SERVER_FIRST) {
          in.read();          // wait for the server's FIN
        }
      }
    }
    Thread.sleep(200);
    report("new connection, " + how.name().toLowerCase()
        .replace('_', ' '), t0, port);
  }
}
```

- **Reuse wins even on loopback:** a new connection per request cost 4.8× as much — no network latency involved. Over a real network add a round trip per connection, and TLS on top.
- **TIME_WAIT lands on whoever closes first** — exactly 2,000 sockets on that side, none on the other. HTTP/1.1 `Connection: close` from a server leaves them on the server.
- **`SO_LINGER(0)` left no TIME_WAIT**: `close()` sends a reset instead of a FIN. It also throws away unsent data and tells the peer something went wrong — a tool for errors, not a fix for port exhaustion.
- **A server that never closes** left 2,000 sockets in CLOSE_WAIT, and the clients' ends waited in FIN_WAIT2 — the one state `tcp_fin_timeout` limits. Growing CLOSE_WAIT in `ss` means a socket leak in the application.
- **One `HttpClient` per application.** The JDK client keeps idle connections for 30 s (`jdk.httpclient.keepalive.timeout`) and reuses them. A new client per request means a new connection per request.

- **Load tests:** open connections the way production does. A generator that reconnects for every request measures handshakes and can exhaust its own ports; one that never reconnects hides the cost of new users.
- **Watch for it:** `ss -tan state time-wait | wc -l` during a test; connection errors such as "Cannot assign requested address" mean the client ran out of ports.
- **Pools need a limit and an idle timeout** shorter than the server's, or the pool hands out connections the server has already closed.

counting sockets by state

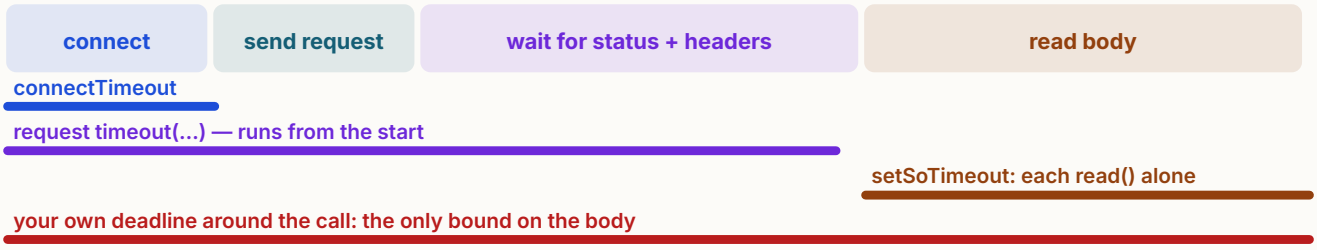
```
static long count(String state, int port, boolean local)
    throws Exception {
  String hex = String.format(":%04X", port);
  long n = 0;
  for (String table : new String[] {"tcp", "tcp6"}) {
    Path p = Path.of("/proc/net/" + table);
    if (!Files.exists(p)) {
      continue;                  // no IPv6 on this machine
    }
    n += Files.readAllLines(p).stream()
        .skip(1).map(l -> l.trim().split("\\s+"))
        .filter(f -> f[3].equals(state))
        .filter(f -> (local ? f[1] : f[2]).endsWith(hex))
        .count();
  }
  return n;
}
```

SOURCES OpenJDK 21 java.net.http module-info.java (jdk.httpclient.keepalive.timeout, default 30) · Linux ip-sysctl.rst · results: deepdive/logs/connections.txt

03 · TIMEOUTS

A timeout bounds one wait, not the call

"The call has a 2-second timeout" is rarely true. Each setting bounds one phase of the call, and the phases nobody set wait forever. Read the JDK's documentation and source carefully, then measure against servers that misbehave in each phase.



PHASE	BOUNDED BY	IF UNSET
TCP connect (handshake)	`HttpClient.Builder.connectTimeout` ; `Socket.connect(addr, ms)`	the kernel's SYN retries
request sent, waiting for the status line and headers	`HttpRequest.Builder.timeout` (counts from the start, connect included)	forever
reading the body	nothing in `HttpClient` ; `Socket.setSoTimeout` bounds each `read()` , not the total	forever
the whole call	only what you add around it — e.g. `future.get(timeout)`	forever

What the request timeout covers

> "If the response is not received within the specified timeout then an HttpTimeoutException is thrown … The effect of not setting a timeout is the same as setting an infinite Duration, i.e. block forever." — `HttpRequest.Builder.timeout`

"The response" means the status line and headers: in JDK 21 the timer is cancelled when `exch.responseAsync()` completes (`MultiExchange` , step "5. handle errors and cancel any timer set"). A body that arrives slowly is not covered.

Defaults: what you get when you set nothing

CLIENT	CONNECT	WAITING FOR DATA	SOURCE
JDK `HttpClient`	none	none ("block forever")	`HttpClient.Builder` , `HttpRequest.Builder` Javadoc
`HttpURLConnection`	0 = infinite	0 = infinite	`URLConnection` : "A timeout of zero is interpreted as an infinite timeout"
`java.net.Socket`	kernel's SYN retries	0 = infinite, per `read()`	`Socket.setSoTimeout` Javadoc
PostgreSQL JDBC	`connectTimeout` 10 s	`socketTimeout` 0 = none	`PGProperty`
HikariCP	`connectionTimeout` 30 s — waiting for a pool connection	—	`HikariConfig`
Lettuce (Redis)	10 s	command timeout 60 s	`SocketOptions` , `RedisURI.DEFAULT_TIMEOUT`
Kafka producer	—	`request.timeout.ms` 30 s	`ProducerConfig`

SOURCES OpenJDK 21: java/net/http/HttpRequest.java, HttpClient.java (Javadoc) · jdk/internal/net/http/MultiExchange.java, ResponseTimerEvent.java · java/net/Socket.java, URLConnection.java (Javadoc) · pgjdbc PGProperty.java · HikariCP HikariConfig.java · Lettuce SocketOptions.java, RedisURI.java · Kafka ProducerConfig.java

03 · TIMEOUTS

Measured: three bad servers, eight settings

Three local peers misbehave in different phases. The harness gives up after 5 s — the overall deadline the HTTP client does not provide. Packets to `192.0.2.1` (an address reserved for documentation, RFC 5737) are dropped with `iptables`, so the handshake simply never completes.

Timeouts.java — one call, two optional timeouts

```
static void call(String name, String hostPort,
    Duration connect, Duration request) throws Exception {
  var client = HttpClient.newBuilder();
  if (connect ≠ null) {
    client.connectTimeout(connect);
  }
  var req = HttpRequest.newBuilder(
      URI.create("http://" + hostPort + "/"));
  if (request ≠ null) {
    req.timeout(request);
  }
  long t0 = System.nanoTime();
  String result;
  try {
    var r = client.build().sendAsync(req.build(),
        BodyHandlers.ofString())
        .get(GIVE_UP_S, TimeUnit.SECONDS);
    result = "returned " + r.statusCode() + ", body "
        + r.body().length() + " bytes";
  } catch (TimeoutException e) {
    result = "still waiting after " + GIVE_UP_S + " s";
  } catch (Exception e) {
    result = e.getCause().getClass().getSimpleName();
  }
  print(name, t0, result);
}
```

PEER	SETTING	WAITED	RESULT
silent: never answers	no timeouts	5.3 s	still waiting after 5 s
silent: never answers	connectTimeout 1 s	5.0 s	still waiting after 5 s
silent: never answers	request timeout 2 s	2.0 s	`HttpTimeoutException`
trickle: headers at once, body 1 byte / 0.5 s	request timeout 2 s	5.0 s	still waiting after 5 s
dropped: packets dropped (no handshake)	no timeouts	5.0 s	still waiting after 5 s
dropped: packets dropped (no handshake)	request timeout 2 s	2.0 s	`HttpConnectTimeoutException`
dropped: packets dropped (no handshake)	connectTimeout 1 s	1.0 s	`HttpConnectTimeoutException`
trickle: headers at once, body 1 byte / 0.5 s	Socket soTimeout 1 s	6.0 s	returned 70 bytes, no error

Real output · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

No timeout at all: the kernel decides

real output · connect() with no timeout

```
java.net.ConnectException: Connection timed out after 136.5 s
```

The kernel's SYN retransmission count, read from `ss` every second: 0 at 1 s · 1 at 2 s · 2 at 3 s · 3 at 4 s · 4 at 5 s · 5 at 6 s · 6 at 8 s · 7 at 12 s · 0 at 18 s · 1 at 19 s · 2 at 20 s · 3 at 21 s · 4 at 22 s · 5 at 23 s · 6 at 25 s · 7 at 29 s · 8 at 32 s · 9 at 36 s · 10 at 69 s. With `tcp_syn_retries = 6` the kernel gave up after **136 s**.

Connect timeouts

> "If the connection cannot be established within the given duration, then HttpClient::send throws an HttpConnectTimeoutException … If a new connection does not need to be established … this timeout duration has no effect."
> — `HttpClient.Builder.connectTimeout`

The request timer fires during connect too: `ResponseTimerEvent` throws `HttpConnectTimeoutException` when the connection is not yet connected, else `HttpTimeoutException`.

- **The trickling body beat the 2 s request timeout**: still waiting at 5 s. A raw socket with a 1 s read timeout read it all in 6 s without an error — every byte came within 1 s.
- **A connect timeout alone does not help** against a server that accepts and stays silent: still waiting at 5 s.
- **Set both, and a deadline:** `connectTimeout` for the handshake, `timeout` for the headers, and a bound on the whole call around it (`future.get(…)`) — the only thing that stopped the trickle.
- **Test each phase** with a fake that misbehaves in it: silent after accept, slow headers, slow body, dropped packets. WireMock's fixed and chunked delays and Toxiproxy do this without writing servers.

SOURCES RFC 5737 (documentation addresses) · Linux ip-sysctl.rst (tcp_syn_retries) · results: deepdive/logs/timeouts.txt, syn.txt, syn-ss.txt

04 · POOLS AND LITTLE'S LAW

Throughput, latency and the size of a pool

Every pool is a queue in front of a resource. Little's Law ties the three numbers you can measure together, for any stable system, whatever the pattern of arrivals: **L = λ·W** — the average number of requests inside equals the rate they leave times the average time each spends inside.

Three consequences

1. A pool caps throughput: with C connections each busy for S seconds, at most C/S requests per second leave. Our database (4 slots, ~10.2 ms per query as measured) tops out near 394 per second.

2. Past the cap, more callers only add waiting: 64 callers in a system that serves 394/s means W = 64 / 394 ≈ 162 ms each, however the queue is arranged.

3. A bigger pool moves the queue — from the pool into the database, where waiting is more expensive: memory, locks, context switches.

How big, then?

> "connections = ((core_count * 2) + effective_spindle_count)" — a starting point from the PostgreSQL project, quoted by HikariCP's "About Pool Sizing"

The reasoning: a CPU core runs one thread at a time; beyond that, more threads add context switching, not throughput — unless they wait on disk or network. Treat the formula as a first guess, then measure throughput and latency as you change it, as below.

The experiment



PoolSize.java — one caller's loop

```
callers[i] = Thread.ofVirtual().start(() → {
  while (System.nanoTime() < end) {
    long t0 = System.nanoTime();
    inSystem.incrementAndGet();
    pool.acquireUninterruptibly();
    long t1 = System.nanoTime();
    db.acquireUninterruptibly();   // the database's queue
    dbWait.add(System.nanoTime() - t1);
    sleep(QUERY_MS);
    db.release();
    pool.release();
    inSystem.decrementAndGet();
    long t2 = System.nanoTime();
    done.add(new Done(t2 - start, t2 - t0));
  }
});
```

Using it: three questions, answered from the measured numbers

QUESTION	LITTLE'S LAW	WITH THIS CHAPTER'S DATABASE
How many connections are busy at a target rate?	L = λ · S (S = time a query holds a connection)	197 queries/s × 10.2 ms = **2.0** connections busy on average — a pool of 4 has headroom
What would a smaller pool deliver?	λ_{max} = C / S	C = 1: **99/s**, C = 2: **197/s** predicted; measured 98/s and 197/s
How long will callers wait once it is saturated?	W = L / λ_{max}	64 callers / 394/s = **162 ms**; measured mean with pool 4: 162.7 ms

S is derived from the saturated rows only (4 slots ÷ 394 queries/s); the smaller pools are then a prediction, checked against their own rows. The law needs averages over a stable period; it says nothing about the tail — p99 needs the distribution (chapter 5).

SOURCES J. D. C. Little, "A Proof for the Queuing Formula: L = λW", Operations Research 9(3), 1961 · HikariCP wiki, "About Pool Sizing" · deepdive/src/.../pool/PoolSize.java

04 · POOLS AND LITTLE'S LAW

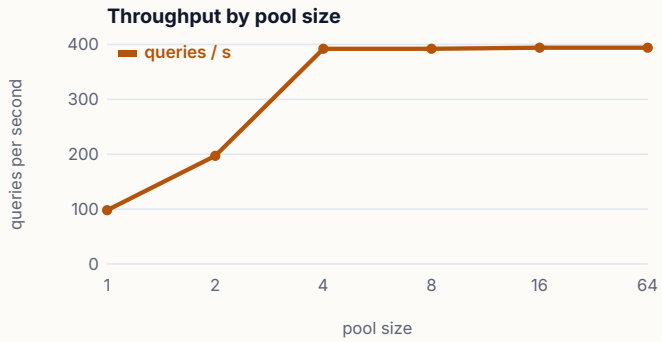
Measured: six pool sizes, then real CPUs

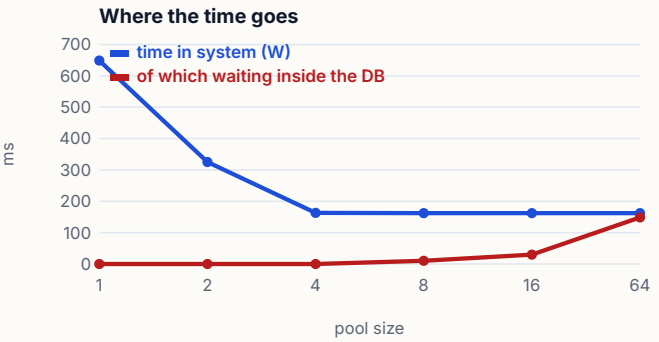
Each row ran for 3 s with 64 callers. λ, W and L were measured independently over the steady window 0.5–2.5 s: λ from completions, W from each request's time in the system, L by counting callers inside every millisecond.

POOL	QUERIES/S (λ)	MEAN MS (W)	P99 MS	WAIT IN DB	L	λ·W
1	98	648.8	659.3	0.0 ms	64.0	63.3
2	197	325.2	327.1	0.0 ms	64.0	64.1
4	392	162.7	163.6	0.0 ms	64.0	63.8
8	392	162.6	163.3	10.1 ms	64.0	63.7
16	394	162.4	162.7	30.3 ms	64.0	64.0
64	394	162.6	163.1	148.6 ms	64.0	64.0

Real output · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

- **Little's Law held in every row:** λ·W was within 1.1% of the measured L.
- **Throughput stopped at the database:** from pool size 4 up, 392–394 queries/s.
- **Bigger pools moved the queue into the database:** the wait inside it grew from 0 to 149 ms per query while the total time stayed the same.
- **Too small a pool** is visible the same way: at 1 and 2 connections the database sat idle while callers queued.



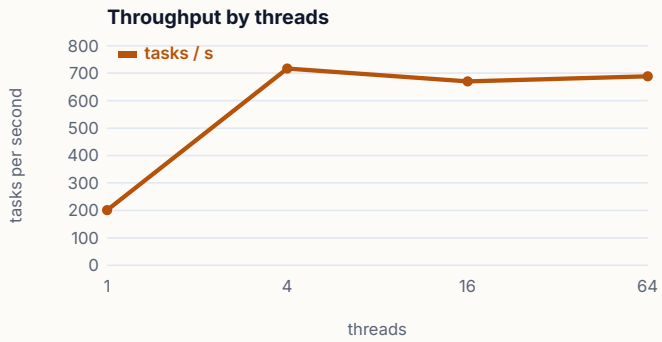


The same question with a real resource: 4 CPU cores

THREADS	TOTAL MS	TASKS/S	TASK P50 MS	TASK P99 MS
1	1995	201	4.9	5.3
4	558	717	5.1	8.0
16	597	670	23.3	34.3
64	580	689	39.2	80.1

400 tasks of about 5 ms of arithmetic each · real output · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

- **Throughput is capped by cores:** 201 tasks/s on 1 thread, 717 on 4, 689 on 64 — no gain past the core count.
- **Each task got slower** as threads were added: p50 4.9 ms alone, 39.2 ms with 64 threads, because the cores were shared. A timeout set from a single-thread test would fail under load.
- **For I/O-bound work** the right size is set by the resource behind it (the database's slots), for CPU-bound work by the cores. Measure both throughput and per-task latency when you change it.



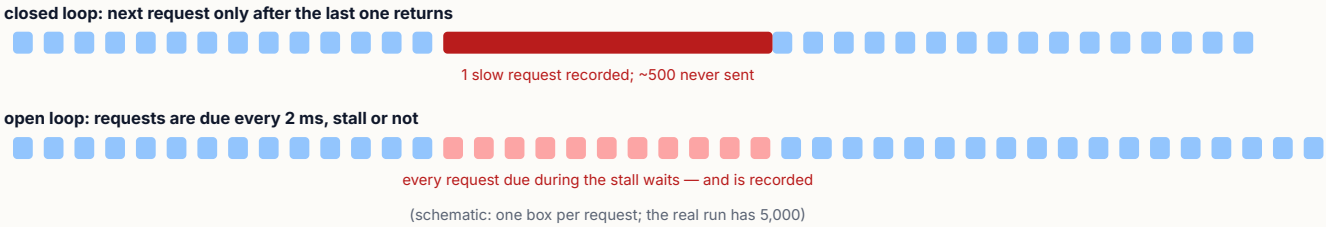


SOURCES results: deepdive/logs/pool.txt, cpupool.txt · deepdive/src/.../pool/PoolSize.java, CpuPool.java

05 · MEASURING LATENCY

Coordinated omission: the load test that hides the stall

Most load tests send a request, wait for the answer, then send the next. When the server stalls, the test stalls with it — and stops sending the requests that real users would have kept sending. Those requests are never measured. Gil Tene named this **coordinated omission**: the tester coordinates with the system it measures, and omits exactly the bad moments.



Two ways to get the truth

Measure from the due time. Keep a schedule — an *open* workload — and count each request's latency from when it should have been sent. wrk2 "measures response latency from the time the transmission *should* have occurred".

Or correct afterwards. HdrHistogram's `recordValueWithExpectedInterval` adds, for a value longer than the expected interval, "an additional series of decreasingly-smaller … value records" — the requests that would have queued behind it.

Open and closed models in load tools

> "In a closed model, the execution time of each iteration dictates the number of iterations executed … In the open model, on the other hand, VUs arrive independently of iteration completion." — k6 documentation

k6: `constant-arrival-rate`, `ramping-arrival-rate` (open). Gatling: `constantUsersPerSec`, `rampUsersPerSec` (open) versus `constantConcurrentUsers` (closed). Real users arrive whether or not the last one finished: test with an open model.

The arithmetic of this chapter's experiment

The program (constants read from `CoordinatedOmission.java`) sends one request every 2 ms for 10 s — 5,000 due — and the server stalls once, for 1000 ms. **Open loop:** 500 requests fall due during the stall, 10% of all; they wait from about 1000 ms down to 0, so everything from about p90 up is stall. **Closed loop:** the client waits with the server, so only one request sees the stall — 1 of 4,501 samples, 0.02%: it can only appear at p99.99 or the maximum. Chapter 5's second page shows exactly that.

Percentiles, read correctly

Stats.java — the percentile every experiment here uses

```java
/** Nearest-rank percentile, in milliseconds. */
public double ms(double percentile) {
  int rank =
      (int) Math.ceil(percentile / 100 * values.length);
  return values[Math.max(rank, 1) - 1] / 1e6;
}
```

p99 of 5,000 samples is the 4,950th smallest — the 50 slowest are above it. Nearest-rank never interpolates: the value reported was really measured.

REQUESTS PER PAGE VIEW	CHANCE ONE OF THEM IS SLOWER THAN P99
1	1%
10	10%
50	39%
100	63%
200	87%

The table is arithmetic, not a measurement: $1 - 0.99^n$. A page that makes 100 backend calls shows a p99 delay to most of its users — so for the calls behind a page, p99.9 and the maximum are the numbers that describe the user's experience.

SOURCES HdrHistogram AbstractHistogram.recordValueWithExpectedInterval (Javadoc) · wrk2 README (github.com/giltene/wrk2) · k6 docs, "Open and closed models" · Gatling docs, "Injection"

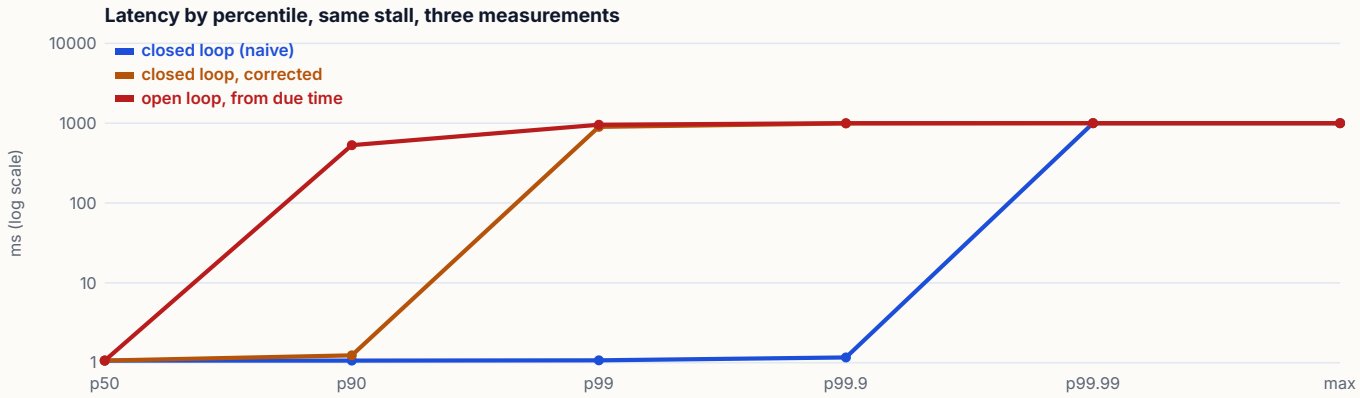
05 · MEASURING LATENCY

Measured: one stall, three reports

A service that answers in 1 ms, except once: from second 2 to second 3 of a 10-second run it stalls. The intended rate is one request every 2 ms (500 per second). The same stall, measured three ways:

MEASUREMENT	SAMPLES	P50 MS	P90 MS	P99 MS	P99.9 MS	P99.99 MS	MAX MS
closed loop — what most tests report	4501	1.06	1.06	**1.07**	1.16	1000.14	1000.14
closed loop, HdrHistogram-style correction	5000	1.06	1.24	900.14	992.14	1000.14	1000.14
open loop, latency from the due time	5000	1.06	530.71	**953.24**	996.40	1000.15	1000.15

Real output · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1



open loop: latency from the due time

```
/** Requests are due every 2 ms; latency from due time. */
static long[] openLoop() throws Exception {
  int total = (int) (RUN_NS / INTERVAL);
  long[] out = new long[total];
  start = System.nanoTime();
  for (int i = 0; i < total; i++) {
    long due = start + i * INTERVAL;
    while (System.nanoTime() < due) {
      Thread.onSpinWait();
    }
    serve();
    out[i] = System.nanoTime() - due;
  }
  return out;
}
```

the correction, as HdrHistogram does it

```
/**
 * HdrHistogram's recordValueWithExpectedInterval: for a value
 * longer than the interval, also record the requests that
 * would have been sent meanwhile: value - k·interval.
 */
static long[] corrected(long[] values) {
  long[] out = new long[values.length * 2];
  int n = 0;
  for (long v : values) {
    out = grow(out, n);
    out[n++] = v;
    for (long m = v - INTERVAL; m ≥ INTERVAL;
        m -= INTERVAL) {
      out = grow(out, n);
      out[n++] = m;
    }
  }
  return Arrays.copyOf(out, n);
}
```

- **The naive p99 was 1.07 ms** — a healthy-looking service. Measured from the due times, p99 was **953.24 ms**: one request in ten was due during the stall, and more waited while the backlog drained.
- **The naive run recorded 4501 samples, not 5,000**: the missing 499 are the requests the client did not send while it waited. The max was right; the percentiles below it were not.
- **The correction recovered most of it** (p99 900.14 ms) from the closed-loop data alone, because the expected interval was known.
- **Report max and high percentiles, with the sample count.** A drop in the count during a test is a stall the percentiles cannot show.
- **Generate load on a schedule** (open model) and measure from the due time; if your tool cannot, correct with the known interval.
- **Never average latencies**, and never average percentiles across machines or intervals: merge the histograms, then read the percentile.

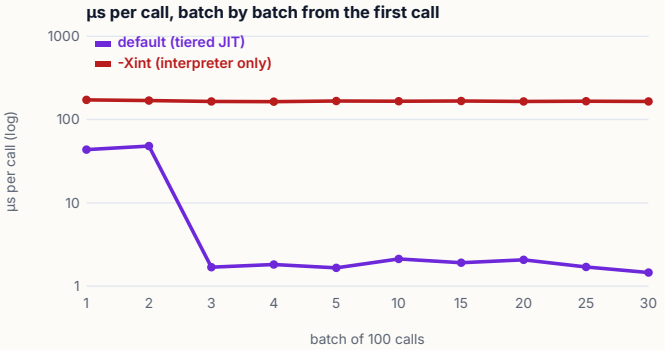
SOURCES results: deepdive/logs/co.txt · deepdive/src/.../latency/CoordinatedOmission.java · G. Tene, “How NOT to Measure Latency” (conference talk)

06 · JIT AND BENCHMARKS

Warm-up: the same method, 87× faster later

HotSpot starts every method in the interpreter, counts calls and loop iterations, and compiles the hot ones — first quickly with C1, collecting a profile, then again with C2 using that profile. Until then the code runs slower, sometimes much slower. A test or benchmark that measures the first calls measures the interpreter.

LEVEL	CODE	USED FOR
0	interpreter	runs bytecode; profiles the method
1	C1, full optimisation, no profiling	trivial methods
2	C1 with invocation and back-edge counters	when C2 is busy
3	C1 with full profiling	the usual first compiled form
4	C2, profile-guided optimisation	the fast steady state



On the runner's JVM: `Tier3InvocationThreshold` = 200, `Tier4InvocationThreshold` = 5000, `Tier4BackEdgeThreshold` = 40000, `CICompilerCount` = 3 — from `-XX:+PrintFlagsFinal`. They are thresholds for counters scaled by the policy, not exact call counts.

WarmUp.java — 30 batches of 100 calls

```
for (int batch = 1; batch <= 30; batch++) {
  long t0 = System.nanoTime();
  for (int i = 0; i < 100; i++) {
    sink = sumOfSquares(DATA);
  }
  double us = (System.nanoTime() - t0) / 1e3 / 100;
  if (batch <= 5 || batch % 5 == 0) {
    System.out.printf("%5d %10.2f%n", batch, us);
  }
}
```

-XX:+PrintCompilation · real output (ms · id · % OSR · level)

```
38    8 %     3       deep.jit.WarmUp::sumOfSquares @ 11 (42
bytes)
39    9       3       deep.jit.WarmUp::sumOfSquares (42
bytes)
39   10 %     4       deep.jit.WarmUp::sumOfSquares @ 11 (42
bytes)
44    8 %     3       deep.jit.WarmUp::sumOfSquares @ 11 (42
bytes)   made not entrant
65  120       4       deep.jit.WarmUp::sumOfSquares (42
bytes)
68    9       3       deep.jit.WarmUp::sumOfSquares (42
bytes)   made not entrant
```

Reading a PrintCompilation line

COLUMN	MEANING
1	milliseconds since the JVM started
2	compile id — the same id for every event of one compilation
3	flags: `%` on-stack replacement · `s` synchronized · `!` has exception handlers · `b` blocking compile · `n` native wrapper
4	tier, 0–4 (table above)
5	the method; `@ 11` after an OSR compile is the bytecode index of the loop it enters at; then the bytecode size
…	`made not entrant`: new calls no longer use this code — a better tier replaced it, or an assumption it made was broken (deoptimisation)

- **Interpreter only:** about 166 µs per call. **Compiled:** about 1.9 µs — 87× faster, the same code.
- **The first batches** ran at 43–48 µs while compilation was under way.
- **Warm up before you measure**, and keep measuring until the numbers are flat; or use JMH, which does both and forks a fresh JVM.
- **Load tests:** discard the first minutes, or plot latency over time — a cold service after a deploy is slower, and that is worth knowing too.

SOURCES OpenJDK 21 hotspot/share/compiler/compilationPolicy.hpp (the five levels), compileTask.cpp (the print format) · -XX:+PrintCompilation, -XX:+PrintFlagsFinal on the runner · results: deepdive/logs/warmup*.txt, jit-flags.txt

06 · JIT AND BENCHMARKS

Measured with JMH: when does the JIT throw your work away?

The JIT may remove code whose result is never used (dead-code elimination) and compute expressions on constants ahead of time (constant folding). A hand-written timing loop cannot tell; JMH can — compare with an empty baseline and hand results back.

DeadCodeBench.java (1/2)

```java
@State(Scope.Thread)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
public class DeadCodeBench {
  static final double CONSTANT = Math.PI;
  static final int[] FIXED = new int[1_000];
  double x = Math.PI;
  int[] data = new int[1_000];

  @Benchmark
  public void baseline() {
    // nothing: the cost of the harness itself
  }

  @Benchmark
  public void discarded() {
    Math.log(x);                    // result unused: dead code
  }

  @Benchmark
  public double returned() {
    return Math.log(x);           // JMH consumes the result
  }

  @Benchmark
  public void blackhole(Blackhole bh) {
    bh.consume(Math.log(x));
  }

  @Benchmark
  public double constant() {
    return Math.log(CONSTANT);    // foldable at compile time?
  }
```

DeadCodeBench.java (2/2)

```java
  @Benchmark
  public void sumDiscarded() {
    long s = 0;
    for (int v : data) {
      s += v;                       // the sum is never used
    }
  }

  @Benchmark
  public long sumReturned() {
    long s = 0;
    for (int v : data) {
      s += v;
    }
    return s;
  }

  @Benchmark
  public void mulDiscarded() {
    double r = x * x + 1;           // never used
  }

  @Benchmark
  public double mulReturned() {
    return x * x + 1;
  }

  @Benchmark
  public double mulConstant() {
    return CONSTANT * CONSTANT + 1;  // static final: foldable
  }
}
```

BENCHMARK	NS/OP	ERROR (99.9%)	MINUS BASELINE
`baseline`	0.335	± 0.005	+0.000
`blackhole`	6.567	± 0.035	+6.232
`constant`	6.540	± 0.056	+6.205
`discarded`	6.573	± 0.038	+6.238
`mulConstant`	0.637	± 0.043	+0.302
`mulDiscarded`	0.335	± 0.001	+0.000
`mulReturned`	0.582	± 0.081	+0.247
`returned`	6.562	± 0.044	+6.227
`sumDiscarded`	0.666	± 0.008	+0.331
`sumReturned`	57.090	± 1.052	+56.755

JMH 1.37 · 1 fork, 3 warm-up and 5 measured iterations of 1 s · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

- **The empty baseline costs 0.34 ns**: the harness itself. Anything at that level did no work.
- **Math.log, result unused**: 6.57 ns; returned 6.56 ns; into a Blackhole 6.57 ns — **not** removed: the unused call still ran. JMH's classic dead-code sample (JMHSample_08, also `Math.log`) does not reproduce here.
- **Math.log of a static final constant**: 6.54 ns — **not** folded here: it cost the same as the call on a field.
- **A pure loop summing 1,000 ints**: 0.67 ns unused, 57.09 ns returned — the JIT **removed** the whole loop.
- `x * x + 1`: 0.34 ns unused, 0.58 ns returned, 0.64 ns on constants — the JIT **removed** the arithmetic.

- **Never trust a benchmark you have not seen fail:** add a baseline and a variant that should be eliminated.
- **Return results or use a Blackhole**, read inputs from `@State` fields, and let JMH fork a fresh JVM (`-f`).

SOURCES JMH 1.37 (org.openjdk.jmh): Blackhole, @State, -f/-wi/-i options; JMHSample_08_DeadCode · results: deepdive/logs/jmh.txt, jmh-full.txt

07 · GC PAUSES AND TAIL LATENCY

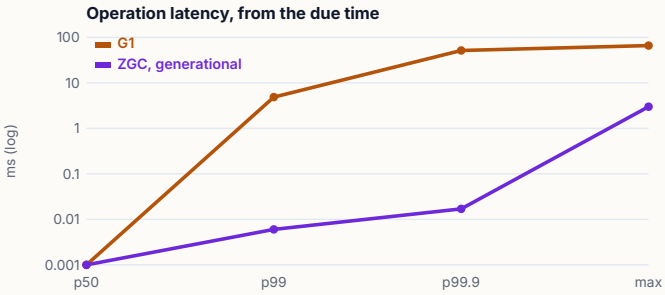
Stop-the-world, or almost never

A garbage collector has to find live objects and make room. Some of that work needs every application thread stopped — a **pause**. G1, the default on a server-class machine, collects in regions and aims for short pauses, but copies live objects while the application waits. ZGC does that work concurrently, while the application runs, and stops it only briefly.

The collectors, as of JDK 21–24

G1: default when the JVM sees at least 2 CPUs and about 1792 MB (chapter 9). Young and mixed collections copy live objects during pauses; marking of the old generation runs concurrently.

Generational ZGC: JEP 439, JDK 21, enabled with `-XX:+UseZGC -XX:+ZGenerational`; the default ZGC mode from JDK 23 (JEP 474); the only mode from JDK 24 (JEP 490). Marking and relocation run concurrently; the remaining pauses do not grow with the heap.



A G1 cycle, as it happened in the measured window

PAUSE	COUNT	LONGEST MS	EVACUATION FAILED	WHAT IT DOES (JDK 21 G1 TUNING GUIDE)
Young (Normal)	5	15.626	–	copies live young objects; promotes survivors to old
Young (Concurrent Start)	2	14.067	–	a young collection that also starts concurrent marking
Remark	2	0.826	–	finishes marking, processes references, frees empty regions
Cleanup	2	0.457	–	decides whether mixed collections follow
Young (Prepare Mixed)	2	13.252	1	the last young-only collection before mixed ones
Young (Mixed)	5	19.381	4	young regions plus some old regions, copied
Full	1	61.895	–	fallback: compacts the whole heap in place, all threads stopped

Evacuation failure (5 of these pauses): G1 could not find free regions to copy live objects into. The tuning guide: G1 then hopes marking and mixed collections catch up; "if this assumption doesn't hold, then G1 will eventually schedule a Full GC" — which is what happened here: one full, in-place compaction of the heap (1024M→405M).

The workload

GcLatency.java — one operation due every 50 µs

```
for (int i = 0; i < total; i++) {
  long due = start + i * INTERVAL_NS;
  while (System.nanoTime() < due) {
    Thread.onSpinWait();
  }
  live[random.nextInt(LIVE)] = new byte[1024];
  sink = new byte[8 * 1024];
  latency[i] = System.nanoTime() - due;
}
```

A live set of 400,000 × 1 KB arrays (≈ 400 MB in a 1 GB heap) that keeps changing — every operation replaces one, so old objects keep dying — plus 8 KB of garbage per operation. One operation is due every 50 µs for 15 s; latency counts from the due time, the open-loop measurement from chapter 5. Pauses come from each JVM's own log (`-Xlog:gc,gc+phases`), from the moment measuring started.

SOURCES JEP 439 (JDK 21), JEP 474 (JDK 23), JEP 490 (JDK 24) · Java SE 21 HotSpot GC Tuning Guide, "Garbage-First (G1) Garbage Collector" · OpenJDK 21 gcConfig.cpp, os.cpp (default selection) · run: GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1, -Xms1g -Xmx1g

07 · GC PAUSES AND TAIL LATENCY

Measured: G1 and generational ZGC, same workload

COLLECTOR	OPERATIONS	P50 MS	P99 MS	P99.9 MS	MAX MS	PAUSES	LONGEST PAUSE MS	PAUSE TOTAL MS
G1	300,000	0.001	4.875	51.136	65.719	19	61.895	205.4
ZGC	300,000	0.001	0.006	0.017	2.973	26	0.021	0.3

Real output and GC logs · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

- **The median was the same** (0.001 ms against 0.001 ms): most operations never meet a pause.
- **The tail was not:** G1's p99.9 was 51.1 ms and its worst operation waited 65.7 ms; with ZGC p99.9 was 0.017 ms and the worst 3.0 ms.
- **The GC logs agree:** G1's longest pause was 61.9 ms; ZGC paused 26 times, the longest 0.021 ms. ZGC's worst operation (3.0 ms) was therefore **not** a GC pause: something else — the OS scheduler, the collector's own threads on 4 CPUs — delayed it.
- **Concurrent is not free:** ZGC uses CPU while the application runs. On a machine with few cores, measure throughput too, not only latency.
- **Log GC in every load test** (`-Xlog:gc*`) and line the pauses up with the slow requests — a slow p99.9 with long pauses at the same moments is GC.
- **Size the heap on purpose** (`-Xmx` or `MaxRAMPercentage`): the default is 25% of the memory the JVM sees, which in a container is the container's limit.

Why p99 sits on the pauses

In an open loop, every operation that falls due during a pause waits for it. G1 was paused for 205.4 ms of the 15 s window: about 4,109 operations (one per 50 µs) were due while the application was stopped — 1.37% of all operations. Anything above that share is inside the tail: p99 is the slowest 1%, p99.9 the slowest 0.1%.

ZGC was paused for 0.34 ms in total — about 7 operations, 0.0023%: too few to reach p99.9.

FULL GC PHASE	MS
Mark live objects	14.581
Prepare compaction	8.136
Adjust pointers	11.121
Compact heap	24.348
Reset Metadata	2.420
pause	**61.895**

The full collection marked and compacted the whole heap while every application thread waited. Without it, G1's longest pause was 19.4 ms.

Every pause in the measured window

G1: 19 pauses

```
 13.399 ms  Young (Normal)
  4.351 ms  Young (Normal)
  7.690 ms  Young (Normal)
 14.888 ms  Young (Normal)
 15.626 ms  Young (Normal)
 14.067 ms  Young (Concurrent Start)
  0.764 ms  Remark
  0.362 ms  Cleanup
 13.252 ms  Young (Prepare Mixed)
 19.381 ms  Young (Mixed)
 17.546 ms  Young (Mixed)
  9.359 ms  Young (Mixed)
  2.455 ms  Young (Concurrent Start)
  0.826 ms  Remark
  0.457 ms  Cleanup
  2.791 ms  Young (Prepare Mixed)
  3.260 ms  Young (Mixed)
  3.059 ms  Young (Mixed)
 61.895 ms  Full
```

ZGC: 26 PAUSES	COUNT	LONGEST MS	WHAT IT DOES
Relocate Start · major, old	3	0.014	starts concurrent relocation
Mark Start · minor	5	0.018	starts concurrent marking
Mark End · minor	5	0.016	ends marking
Relocate Start · minor	5	0.011	starts concurrent relocation
Mark Start · major, young	2	0.021	starts concurrent marking
Mark End · major, young	2	0.016	ends marking
Relocate Start · major, young	2	0.012	starts concurrent relocation
Mark End · major, old	2	0.015	ends marking

The log tags each ZGC pause `y:` (a minor cycle: young generation only), or `Y:` / `O:` (the young and old generations of a major cycle, which it marks `(Major)`).

SOURCES Java SE 21 HotSpot GC Tuning Guide (G1 cycle, evacuation failure) · results: deepdive/logs/gc-G1.txt, gc-G1.log, gc-ZGC.txt, gc-ZGC.log

08 · VIRTUAL THREADS

Cheap threads, and the one thing that pins them

A virtual thread (JEP 444, final in JDK 21) is a Java thread that is not tied to an OS thread. It runs *mounted* on a carrier — a platform thread of a small pool — and when it blocks on I/O, a sleep or a lock, it *unmounts* and the carrier runs another one. Blocking becomes cheap; you can have a thread per task again.

Carriers

The scheduler is a `ForkJoinPool` whose parallelism is `availableProcessors()` unless `jdk.virtualThreadScheduler.parallelism` says otherwise (JDK 21 `VirtualThread.java`). The runner had 4.

Pinning (JDK 21–23)

Blocking inside `synchronized` keeps the virtual thread **pinned**: it cannot unmount, so it holds its carrier while it waits. JEP 491 (JDK 24) lets virtual threads unmount inside `synchronized`; the `jdk.tracePinnedThreads` property used here was removed with it.

VirtualThreads.java — the same wait, two locks

```
static void blockInSynchronized() {
  Object lock = new Object();             // one lock per task
  synchronized (lock) {
    sleep(100);                           // pins the carrier
  }
}

static void blockInLock() {
  var lock = new ReentrantLock();
  lock.lock();
  try {
    sleep(100);                           // unmounts
  } finally {
    lock.unlock();
  }
}
```

RUN	WALL TIME
10,000 tasks, 200 platform threads	5.03 s
10,000 tasks, virtual thread each	0.19 s
100 tasks, synchronized + sleep	2.51 s
100 tasks, ReentrantLock + sleep	0.10 s

Real output · JDK 21 · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

-Djdk.tracePinnedThreads=short · real output

```
VirtualThread[#10229]/runnable@ForkJoinPool-1-worker-4
reason:MONITOR

deep.vt.VirtualThreads.blockInSynchronized(VirtualThreads.ja
va:51) ⟸ monitors:1
```

- **10,000 blocking tasks:** 5.03 s on 200 platform threads (10,000 / 200 × 0.1 s = 5 s), 0.19 s with a virtual thread each.
- **Pinned:** 100 tasks holding a monitor while sleeping took 2.51 s = 100 / 4 carriers × 0.1 s. With `ReentrantLock`: 0.10 s.
- **On JDK 21–23, replace `synchronized` around blocking calls** with `java.util.concurrent` locks, or upgrade: JDK 24 removes this kind of pinning.
- **Virtual threads do not make CPU work faster** — chapter 4's CPU pool applies unchanged. They make waiting cheap.

What a blocking call does to the carrier, JDK 21

THE VIRTUAL THREAD BLOCKS IN…	CARRIER	EVIDENCE
`Thread.sleep`, `LockSupport.park`, `java.util.concurrent` locks and queues, socket I/O	**released** — the virtual thread unmounts	measured above: ReentrantLock + sleep, 0.10 s
`Object.wait()`, file I/O such as `FileInputStream.read`	held, but **compensated**: the pool adds a carrier for the duration	JDK 21 `Object.wait` and `FileInputStream.read` call `Blocker.begin()` → `ForkJoinPool` compensated block
any of the above inside `synchronized`	**pinned**, not compensated: the carrier waits too	measured above: 2.51 s = 100 / 4 × 0.1 s; `VirtualThread.parkOnCarrierThread`
a native method or foreign function on the stack	**pinned**	JEP 444, "Pinning"

Compensation has a ceiling: `jdk.virtualThreadScheduler.maxPoolSize`, by default the larger of the parallelism and 256 (JDK 21 `VirtualThread.createDefaultScheduler`). Thousands of virtual threads in file I/O at once still queue.

Finding pins in production: the JFR event `jdk.VirtualThreadPinned` is on in the default JFR settings, with a 20 ms threshold and a stack trace (JDK 21 `default.jfc`). Do not pool virtual threads — JEP 444: they are cheap enough to create one per task.

SOURCES JEP 444 (JDK 21), JEP 491 (JDK 24) · OpenJDK 21 java/lang/VirtualThread.java, Object.java, io/FileInputStream.java, jdk/internal/misc/Blocker.java · jdk.jfr default.jfc · results: deepdive/logs/vt.txt

09 · CONTAINERS AND CPUS

What the JVM sees inside a container

A container limit is not a smaller machine. It is a cgroup setting: a CPU **quota** per period, and a memory ceiling. The JVM reads both at start-up (container support has been on by default since JDK 10) and sizes itself from them — the number of CPUs it reports, the heap, the garbage collector, and every thread pool sized from `availableProcessors()`.

CPUs from the quota

`--cpus=N` writes `cpu.max` as *quota period*. HotSpot divides quota by period and **rounds up**, capped at the host's CPUs (JDK 21 `cgroupUtil_linux.cpp`: `ceilf(quota / period)`). `-XX:ActiveProcessorCount` overrides the result.

Heap from the memory limit

Without `-Xmx`, the maximum heap is `MaxRAMPercentage` of the memory the JVM sees — **25** by default (`gc_globals.hpp`). `MinRAMPercentage` (50) is used instead only when memory is so small that half of it is below the default `MaxHeapSize` (`arguments.cpp`).

The collector from both

G1 is chosen only on a "server-class machine": at least **2** active processors and at least **2 GB − 256 MB** of memory (`os.cpp` `is_server_class_machine`, with an extra check for hyper-threaded packages). Otherwise **Serial** — one thread, stop-the-world (`gcConfig.cpp`).

Measured: the same JVM, different limits

DOCKER RUN …	CPU.MAX	MEMORY.MAX	CPUS	MAX HEAP	HEAP / LIMIT	COMMON POOL	GC
`no limits`	max/100000	max	**4**	3,998 MB	–	3	**G1**
`--cpus=2 --memory=2g`	200000/100000	2,048 MB	**2**	512 MB	25.0%	1	**G1**
`--cpus=1 --memory=1g`	100000/100000	1,024 MB	**1**	247 MB	24.1%	1	**Serial**
`--cpus=0.5 --memory=512m`	50000/100000	512 MB	**1**	123 MB	24.0%	1	**Serial**
`--cpus=1 --memory=1g -XX:ActiveProcessorCount=4`	100000/100000	1,024 MB	**4**	247 MB	24.1%	3	**Serial**
`--cpus=1 --memory=1g -XX:MaxRAMPercentage=75`	100000/100000	1,024 MB	**1**	742 MB	72.5%	1	**Serial**

eclipse-temurin:21-jdk on GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1 · the program prints `/sys/fs/cgroup` and what the JVM decided · GC names: Copy + MarkSweepCompact is Serial

Resources.java

```
public static void main(String[] args) throws Exception {
  var rt = Runtime.getRuntime();
  String gcs = ManagementFactory.getGarbageCollectorMXBeans()
      .stream().map(b -> b.getName())
      .collect(Collectors.joining(", "));
  System.out.printf("cgroup cpu.max %-14s memory.max %-11s"
      + " | CPUs %d  max heap %4d MB  commonPool %d  GC %s%n",
      cgroup("cpu.max"), cgroup("memory.max"),
      rt.availableProcessors(), rt.maxMemory() / (1 << 20),
      ForkJoinPool.getCommonPoolParallelism(), gcs);
}
```

- **--cpus=0.5** is reported as **1** CPU: 0.5 rounded up. The JVM cannot express half a CPU; the kernel enforces it by throttling.
- **2 CPUs and 2 GB** crossed the server-class line: **G1**, heap 512 MB. **1 CPU and 1 GB** did not: **Serial**.
- **ActiveProcessorCount=4** made the JVM report 4 CPUs (common pool 3) — and it still chose **Serial**: 1 GB is below the memory line.
- **MaxRAMPercentage=75** raised the heap from 247 MB to 742 MB in the same 1 GB container.

A quota is time, not cores

`--cpus=0.5` lets the container run for half of each scheduling period, then stops it until the next one. The JVM still reports 1 CPU and runs one GC thread. A multi-threaded burst can use the whole quota early in the period and then wait — throttling, visible in `cpu.stat` as `nr_throttled` and `throttled_usec`.

SOURCES OpenJDK 21: os/linux/cgroupUtil_linux.cpp, runtime/os.cpp, gc/shared/gcConfig.cpp, gc/shared/gc_globals.hpp, runtime/arguments.cpp · results: deepdive/logs/containers.txt

09 · CONTAINERS AND CPUS

Everything sized from availableProcessors()

The CPU count the JVM reports is read once and used everywhere. A pod limited to one CPU gets one-thread defaults across the JDK and its libraries; a limit of 0.5 looks exactly like a limit of 1, but runs half the time.

SIZED FROM THE CPU COUNT	DEFAULT	SOURCE	MEASURED (HOST / 2 / 1 CPU)
`ForkJoinPool.commonPool()` — parallel streams, `CompletableFuture.*Async`	CPUs − 1, at least 1	JDK 21 `ForkJoinPool.java`	3 / 1 / 1
virtual-thread carriers	CPUs	JDK 21 `VirtualThread.java`	chapter 8
G1/Parallel GC worker threads (`ParallelGCThreads`)	CPUs up to 8, then 8 + 5/8 of the rest	JDK 21 `workerPolicy.cpp`	–
Netty event-loop threads	2 × CPUs	Netty 4.1 `MultithreadEventLoopGroup`	–
the garbage collector itself	Serial below 2 CPUs or 1792 MB	`os.cpp` , `gcConfig.cpp`	Serial at 1 CPU

The heap is not the whole process

The memory limit applies to everything the process uses: the heap, plus metaspace, the code cache, thread stacks, direct buffers and the collector's own structures. Raising `MaxRAMPercentage` to 75 (measured above: 742 MB of 1,024 MB) leaves the rest for all of that; go higher only after measuring native memory with `-XX:NativeMemoryTracking=summary` and `jcmd <pid> VM.native_memory` .

- **Run performance tests with the production limits** — the same `--cpus` and `--memory` . A laptop or an unlimited runner chooses a different collector, a different heap and different pool sizes.
- **Print what the JVM decided** at start-up: `-Xlog:gc*` names the collector and heap; `java -XX:+PrintFlagsFinal -version` inside the container shows every ergonomic choice.
- **Give a JVM service at least 2 CPUs and 2 GB**, or choose the collector yourself (`-XX:+UseG1GC`) — otherwise a small pod gets Serial.
- **Set the heap on purpose** — `-Xmx` , or `MaxRAMPercentage` for a heap that follows the limit.

Measured: throttling

DOCKER RUN …	BUSY THREADS	LATE P50 MS	LATE P99 MS	LATE MAX MS	PERIODS	THROTTLED PERIODS
`no limit`	0	0.1	0.1	**0.2**	–	–
`--cpus=0.5`	0	0.1	0.1	**0.2**	13	0
`no limit`	3	0.1	0.4	**0.9**	–	–
`--cpus=2`	3	0.1	0.6	**25.7**	31	31
`--cpus=0.5`	3	0.1	84.8	**85.1**	48	48

Throttle.java: busy threads use the quota while a timer thread sleeps 5 ms, 600 times, and records how late it wakes; periods and throttling from the container's `cpu.stat` · GitHub Actions · Linux 6.17.0-1022-azure · 4 CPUs · JDK 21.0.12.1

- **--cpus=0.5 with 3 busy threads:** 48 of 48 periods throttled, and a 5 ms sleep woke up to **85 ms** late — against 0.9 ms with no limit.
- **The timer thread did nothing wrong:** it needed microseconds of CPU, but once the container's quota was spent no thread in it could run until the next period. In a service, that is a request stalled for the rest of the period.

SOURCES OpenJDK 21: java/util/concurrent/ForkJoinPool.java, java/lang/VirtualThread.java, gc/shared/workerPolicy.cpp · Netty 4.1 MultithreadEventLoopGroup.java · Linux cgroup v2 documentation (cpu.max, cpu.stat) · results: deepdive/logs/containers.txt, throttle.txt

10 · THE JAVA MEMORY MODEL

What another thread is allowed to see

Without synchronisation, Java promises much less than the source code suggests. The compiler and the CPU may reorder and cache memory accesses as long as a single thread cannot tell. The Java Memory Model (JLS chapter 17) says exactly when another thread is guaranteed to see a write — and that guarantee is the *happens-before* order.

Data race (JLS §17.4.5)

Two accesses to the same variable **conflict** if at least one is a write. A program has a **data race** when two conflicting accesses are not ordered by happens-before.

The guarantee (§17.4.3): "If a program has no data races, then all executions of the program will appear to be sequentially consistent." With a race, the JMM allows results no interleaving of the source lines can produce.

Happens-before, JLS §17.4.5 (quoted)

- An unlock on a monitor *happens-before* every subsequent lock on that monitor.
- A write to a `volatile` field *happens-before* every subsequent read of that field.
- A call to `start()` on a thread *happens-before* any actions in the started thread.
- All actions in a thread *happen-before* any other thread successfully returns from a `join()` on that thread.

Plus program order within a thread, and transitivity. `java.util.concurrent` documents the same edges for its locks, queues, atomics and futures.

Measured with jcstress: store buffering

StoreBufferingTest.java (outcomes declared with @Outcome)

```
@State
public class StoreBufferingTest {
  int x;
  int y;

  @Actor
  public void first(II_Result r) {
    x = 1;
    r.r1 = y;
  }

  @Actor
  public void second(II_Result r) {
    y = 1;
    r.r2 = x;
  }
}
```

Plain fields — every interleaving of the four lines gives at least one 1. Yet:

RESULT	SAMPLES	SHARE	EXPECTED	MEANING
`0, 0`	168,673,148	16.49%	Interesting	both read before either write was visible
`0, 1`	434,408,475	42.46%	Acceptable	some interleaving of the four steps
`1, 0`	420,032,698	41.05%	Acceptable	some interleaving of the four steps
`1, 1`	40,387	<0.01%	Acceptable	some interleaving of the four steps

The same with `volatile int x, y` — `0, 0` declared forbidden:

RESULT	SAMPLES	SHARE	EXPECTED	MEANING
`0, 0`	0	0.00%	Forbidden	both read before either write was visible
`0, 1`	303,293,906	45.27%	Acceptable	some interleaving of the four steps
`1, 0`	298,009,901	44.48%	Acceptable	some interleaving of the four steps
`1, 1`	68,714,261	10.26%	Acceptable	some interleaving of the four steps

- **168,673,148 of 1,023,154,708 plain runs (16.49%) ended 0, 0**: both threads read before either write was visible. No interleaving of the source allows that; the JMM does, because the program has a data race.
- **With volatile: 0 of 670,018,068**. Volatile reads and writes are synchronisation actions, totally ordered (§17.4.4), so one write must come first and the other thread's read must see it.
- **Where the reordering comes from:** a store can wait in the CPU's store buffer while a later load goes ahead (x86 allows this one reordering, Intel SDM Vol. 3A, "Loads May Be Reordered with Earlier Stores to Different Locations"), and the JIT may reorder too — jcstress runs each test interpreted, with C1 and with C2, and with C2's code-motion randomisers.
- **The machine:** 4 CPUs (INTEL(R) XEON(R) PLATINUM 8573C); jcstress pins each actor thread to its own CPU.

SOURCES JLS SE 21 §17.4.2–17.4.5 · jcstress 0.16 (org.openjdk.jcstress) · Intel 64 and IA-32 Architectures SDM, Vol. 3A, memory ordering · results: ci-logs-ops logs/jcstress.log

10 · THE JAVA MEMORY MODEL

Visibility: a loop that never sees the flag

Happens-before is also about *whether* a write is seen at all. A thread spinning on a plain field has no happens-before edge with the thread that sets it, so the JIT may read the field once and keep the value. JLS §17.3 says so in nearly these words: "The compiler is free to read the field `this.done` just once, and reuse the cached value in each execution of the loop."

StaleFlag.java — the loop runs 1 s, then the flag is set

```
static String stops(boolean useVolatile) throws Exception {
  Thread spinner = new Thread(() -> {
    if (useVolatile) {
      while (!marked) {
        // spin
      }
    } else {
      while (!plain) {
        // spin
      }
    }
  });
  spinner.setDaemon(true);
  spinner.start();
  Thread.sleep(1_000);      // let the JIT compile the loop
  if (useVolatile) {
    marked = true;
  } else {
    plain = true;
  }
  spinner.join(3_000);
  return spinner.isAlive()
      ? "still spinning 3 s after the flag was set"
      : "stopped";
}
```

real output · 5 runs in CI

```
plain field:    still spinning 3 s after the flag was set
volatile field: stopped
plain field:    still spinning 3 s after the flag was set
volatile field: stopped
plain field:    still spinning 3 s after the flag was set
volatile field: stopped
plain field:    still spinning 3 s after the flag was set
volatile field: stopped
plain field:    still spinning 3 s after the flag was set
volatile field: stopped
```

- **Plain field: 5 of 5 runs** still spinning 3 s after the flag was set — the compiled loop never reads the field again.
- **Volatile: 5 of 5** stopped. Every volatile read must observe the latest write in the synchronisation order.

The same question, asked by jcstress

SpinTest.java — Mode.Termination

```
@JCStressTest(Mode.Termination)
@Outcome(id = "TERMINATED", expect = ACCEPTABLE,
    desc = "saw it")
@Outcome(id = "STALE", expect = ACCEPTABLE_INTERESTING,
    desc = "spins forever")
@State
public class SpinTest {
  boolean ready;

  @Actor
  public void spin() {
    while (!ready) {
      // wait
    }
  }

  @Signal
  public void signal() {
    ready = true;
  }
}
```

RESULT	SAMPLES	SHARE	EXPECTED	MEANING
`STALE`	2	0.12%	Interesting	spins forever
`TERMINATED`	1,615	99.88%	Acceptable	saw it

With `volatile boolean ready`: `STALE` 0 of 3,222, declared forbidden.

- **Only 2 of 1,617 runs (0.12%) got stuck.** jcstress signals almost immediately, often before the loop is compiled; StaleFlag waits a second so that it is. A race that shows up 0.12% of the time in a stress tool shows up in production under load.
- **Lost updates**, from the guide's own counter tests: `count++` on a plain field lost an update in 9.36% of 1,301,273,108 runs; `AtomicInteger` in 0 of 648,217,108.

- **Share mutable state through something that creates the edge:** `volatile`, an `Atomic*`, a lock, a concurrent collection, a future's result, `Thread.start` / `join`.
- **Publish immutable objects through final fields** (JLS §17.5): a correctly constructed object's final fields are seen initialised by every thread.
- **A passing unit test proves nothing here** — the bad outcome may need the JIT, another CPU, or timing a test never hits. Use jcstress for the critical pieces.
- **volatile gives visibility and ordering, not atomicity:** `volatileCount++` is still read, add, write.

SOURCES JLS SE 21 §17.3, §17.4, §17.5 · jcstress 0.16: Mode.Termination, @Signal · results: deepdive/logs/staleflag.txt; ci-logs-ops logs/jcstress.log

BACK MATTER

Sources

Everything this book relies on for how things work. Measured results come from the logs listed last.

Standards and specifications

- RFC 9293 — Transmission Control Protocol (2022): Nagle (§3.7.4), delayed ACK (§3.8.6.3), TIME-WAIT and MSL (§3.4.2–3.4.3), states (§3.3.2)
- RFC 896 — J. Nagle, Congestion Control in IP/TCP Internetworks (1984)
- RFC 5737 — IPv4 address blocks reserved for documentation (192.0.2.0/24)
- JLS SE 21, chapter 17 — threads and locks; the memory model (§17.3–17.5)
- JEP 439 Generational ZGC · JEP 444 Virtual threads · JEP 474 ZGC generational by default · JEP 490 non-generational ZGC removed · JEP 491 synchronized without pinning

Linux 6.17 source and documentation

- include/net/tcp.h — TCP_DELACK_MIN/MAX, TCP_MAX_QUICKACKS, TCP_TIMEWAIT_LEN
- net/ipv4/tcp_input.c (quick ACKs, pingpong mode) · net/ipv4/tcp_output.c (tcp_event_data_sent)
- Documentation/networking/ip-sysctl.rst — tcp_pingpong_thresh, tcp_tw_reuse, tcp_fin_timeout, tcp_syn_retries, ip_local_port_range
- Documentation/admin-guide/cgroup-v2.rst — cpu.max, cpu.stat, memory.max

OpenJDK 21 source

- java.net.http: PlainHttpConnection, MultiExchange, ResponseTimerEvent, module-info (keep-alive 30 s)
- java.base: StandardSocketOptions, Socket, URLConnection, VirtualThread, Object, FileInputStream, Blocker, ForkJoinPool
- jdk.httpserver ServerConfig · jdk.net ExtendedSocketOptions · jdk.jfr default.jfc
- HotSpot: compilationPolicy.hpp, compileTask.cpp, os.cpp, gcConfig.cpp, gc_globals.hpp, arguments.cpp, workerPolicy.cpp, cgroupUtil_linux.cpp

Documentation

- Java SE 21 HotSpot GC Tuning Guide — the G1 cycle, evacuation failure, full GC
- HttpClient.Builder and HttpRequest.Builder Javadoc — connect and request timeouts
- Intel 64 and IA-32 Architectures SDM, Vol. 3A — memory ordering
- HikariCP wiki, “About Pool Sizing” · J. D. C. Little, “A Proof for the Queuing Formula: L = λW”, Operations Research 9(3), 1961

Libraries (source, default branch or named version)

- Tomcat SocketProperties · Jetty 12.0 ServerConnector · Netty 4.1 DefaultSocketChannelConfig, MultithreadEventLoopGroup
- pgjdbc PGProperty · HikariCP HikariConfig · Lettuce SocketOptions, RedisURI · Kafka Selector, ProducerConfig

Tools

- JMH 1.37 · jcstress 0.16 · HdrHistogram (recordValueWithExpectedInterval) · wrk2 · k6, open and closed models · Gatling, injection profiles

The evidence in this repository

- Experiments: `deepdive/src/main/java/deep/` and `concurrency/src/main/java/shop/` (jcstress) · runner script `deepdive/run.sh`
- Logs: branch `ci-logs-deepdive` (`logs/`) and `ci-logs-ops` (`logs/jcstress.log`), rewritten on every push to main

FIND ANYTHING

Index A–Z

A

ActiveProcessorCount 19
availableProcessors() 19, 20

B

Blackhole 15

C

Carrier threads 18
cgroups 19, 20
CLOSE_WAIT 06, 07
Connect timeout 08, 09
Constant folding 15
Coordinated omission 12, 13
CPU throttling 19, 20
CPU-bound pools 11
--cpus / cpu.max 19, 20

D

Data race 21
Dead-code elimination 15
Delayed ACK 04, 05
Deoptimisation 14

E

Ephemeral ports 06
Evacuation failure 16, 17

F

FIN_WAIT2 06, 07
Final fields 22
ForkJoinPool.commonPool 20
Full GC 16, 17

G

G1 16, 17
GC logging 16, 17

H

Happens-before 21, 22
HdrHistogram 12
HikariCP 10, 08

J

Java Memory Model 21, 22
jcstress 21, 22
jdk.VirtualThreadPinned (JFR) 18
JIT tiers (C1, C2) 14
JMH 15

K

Keep-alive (HTTP) 06, 07

L

Little's Law 10, 11
Lost update 22

M

MaxRAMPercentage 19, 20

N

Nagle's algorithm 04, 05
Native memory 20

O

On-stack replacement 14
Open vs closed loop 12, 13

P

Percentiles 12, 13
Pingpong mode (Linux) 04
Pinning 18
Pool sizing 10, 11
PrintCompilation 14

R

ReentrantLock 18
Request timeout (HttpClient) 08, 09

S

Serial GC 19
Server-class machine 19, 16
SO_LINGER / RST 07
SO_TIMEOUT / read timeout 08, 09
Store buffering 21
SYN retries 09
synchronized 18, 21

T

Tail latency 17, 13
TCP_NODELAY 04, 05
TCP_QUICKACK 04, 05
tcp_tw_reuse 06
tcpdump 04, 06
Three-way handshake 06
TIME_WAIT 06, 07
Timeout defaults 08

V

Virtual threads 18
Visibility 22
volatile 21, 22

W

Warm-up 14
wrk2 / k6 / Gatling 12

Z

ZGC (generational) 16, 17

Three ways in

A request is slow

Nagle **p.04**, connection reuse **p.07**, pools **p.10**, GC pauses **p.17**, CPU throttling **p.20**.

A request hangs

Timeouts and their defaults **p.08**, the three broken peers **p.09**, pinned virtual threads **p.18**, a loop that never sees a flag **p.22**.

The numbers look wrong

Coordinated omission **p.12**, warm-up **p.14**, JMH and dead code **p.15**, production limits in tests **p.19**.